Spark Compiler

Submitted By

Student Name	Student ID
Md. Sazzad Sabbir Sun	241-15-882
Didarul Islam	241-15-214
Tushar Abdullah	241-15-130
Md. Sanjidor Rahman Shimul	241-15-956

LAB PROJECT REPORT

This Report Presented in Partial Fulfillment of the course

CSE314: Compiler Design Lab in the Department of Computer Science and Engineering

DAFFODIL INTERNATIONAL UNIVERSITY

Dhaka, Bangladesh

05/04/2026

DECLARATION

We hereby declare that this lab project has been done by us under the supervision of **Mushfiqur Rahman**, **Assistant Professor**, Department of Computer Science and Engineering, Daffodil International University. We also declare that neither this project nor any part of this project has been submitted elsewhere as lab projects.

Submitted To:

_____

Mushfiqur Rahman
Assistant Professor
Department of Computer Science and Engineering
Daffodil International University

Submitted by

<table>
<tr><td colspan="2">_____
Md. Sazzad Sabbir Sun
ID: 241-15-882
Dept. of CSE, DIU</td></tr>
<tr><td>_____
Didarul Islam
ID: 241-15-214
Dept. of CSE, DIU</td><td>_____
Tushar Abdullah
ID: 241-15-130
Dept. of CSE, DIU</td></tr>
<tr><td>_____
Md. Sanjidor Rahman
ID: 241-15-956
Dept. of CSE, DIU</td><td>_____
Student Name Student
ID:
Dept. of CSE, DIU</td></tr>
</table>

COURSE & PROGRAM OUTCOME

The following course have course outcomes as following:

Table 1: Course Outcome Statements

CO's	Statements
CO1	**Understand** the working procedure of a compiler for debugging programs.
CO2	**Analyze** the role of syntax and semantics of Programming languages in compiler construction.
CO3	**Apply** the techniques, algorithms, and different tools used in Compiler Construction in the design and construction of the phases of a compiler's components
CO4	**Create** a project by explaining complex computer engineering activities with the computer engineering community by performing effective communication through demonstration and presentation

Table 2: Mapping of CO, PO, Blooms, KP and CEP

CO	PO	Blooms	KP	CEP	CEA
CO1	PO1	C1, C2, A1, A2	K4- K6	EP1	
CO2	PO2	C3, C4, A2, P3	K6	EP2, EP4	
CO3	PO3	C4, A2	K6		
CO4	PO10	C6, A2, A4, P6, P7	K7-K10		EA1, EA2

The mapping justification of this table is provided in section **4.3.1**, **4.3.2** and **4.3.3**.

Table of Contents

Chapter 1

Introduction

This chapter provides an overview of the Spark Compiler project, including its objectives, scope, and significance. It sets the foundation by explaining the problem domain and the goals the project aims to achieve.

1.1 Introduction

Th Programming languages are used to give instructions to a computer so that the computer can perform tasks. There are many programming languages such as C, Java, and Python. These languages are powerful, but they are complex. Beginners often use these languages but do not understand how these languages actually work inside a compiler.

In compiler design courses, students learn many topics such as lexical analysis, syntax analysis, semantic analysis, intermediate code generation, and code generation. But most students only learn the theory. They do not see how a real compiler works step by step.

The problem is that it is difficult to understand compiler design without building a real compiler. So, we decided to create a simple programming language called “Spark”. The purpose of Spark is not to replace other programming languages, but to help students understand how a compiler works.

Spark is a simple programming language that supports basic features like variable declaration, arithmetic operation, follow precedence order, if statement, while loop, post increment. The Spark compiler reads the source code, checks for errors, and then generates intermediate code and target code. This project shows all the phases of a compiler in a simple way.

1.2 Motivation

The main motivation of this project is to understand compiler design in a practical way. Many compiler topics are difficult to understand by reading books only. When we build a compiler ourselves, we can understand each phase clearly.

Another motivation is to create a simple language for learning. Many programming languages are large and complex. Beginners get confused by complex syntax and rules. Spark is designed to be simple and easy so that students can focus on compiler concepts instead of complex syntax.

This project also helps us learn practical skills such as compiler design, tokenization, parsing, code generation, program solving and software designing.

1.3 Objectives

The main objectives of this project are:

1. To design a simple programming language called Spark.
2. To build a lexical analyzer to break the source code into tokens.
3. To build a syntax analyzer (parser) to check grammar rules.
4. To perform semantic analysis to check meaning and errors.

5. To generate intermediate code (Three Address Code).
6. To generate target code from intermediate code.
7. To detect and show error messages.
8. To show all phases of a compiler in one project.
9. To implement the project using Node.js.
10. To create a working mini compiler.

1.4 Feasibility Study

Before developing the Spark language, we studied existing programming languages and compiler projects. Many programming languages such as C, Java, and Python use compilers or interpreters to convert source code into machine code. There are also many educational compiler projects that demonstrate compiler phases such as lexical analysis, parsing, and code generation.

We also studied simple scripting languages and mini-compiler projects available online. These projects helped us understand how a programming language is designed and how source code is processed step by step.

From this study, we concluded that developing a simple programming language is feasible using modern tools such as Node.js. Node.js provides file handling, string processing, and modular programming features which make compiler implementation easier.

Therefore, the Spark language project is technically feasible and can be implemented within the given time using available tools and knowledge.

1.5 Gap Analysis

After studying existing programming languages and compiler projects, we found some problems.

Many programming languages are very complex. They are not designed for learning compiler design. Many compiler projects only show one part of the compiler, such as lexical analysis or parsing. They do not show the full compiler.

Gaps are:

1. There are not many simple languages made for learning compiler design.
2. Many projects do not show all compiler phases in one system.
3. Students need a simple and complete compiler project for learning.
4. Existing languages are too complex for beginners to understand compiler internals.

Our Spark project tries to solve these problems by creating a simple language and a complete compiler that shows all phases.

1.6 Project Outcome

The Spark project helps students and beginners understand how a compiler works in a simple way. By building and using Spark, we can see every step of converting source code into target code. The expected outcomes of this project are:

1. A simple programming language called Spark.
2. A lexical analyzer that converts source code into tokens.
3. A syntax analyzer that checks grammar.
4. A semantic analyzer that checks errors.
5. Intermediate code generation (Three Address Code).

6. Target code generation.
7. Error detection and error messages.
8. A working mini compiler.
9. A full project report and documentation.
10. A project that helps students learn compiler design easily.

Chapter 2

System Architecture

This chapter describes the overall system design and architecture of the Spark Compiler project. It outlines the major components, their interactions, and the technologies used to build an efficient and scalable system.

2.1 Requirement Analysis & Design Specification

This section describes what the Spark system needs to do and how it is designed. It defines the functional and non-functional requirements of the compiler and presents the structure of the system through diagrams. The goal is to provide a clear idea about the Spark system's design before implementation.

2.1.1 Requirements

This subsection lists the requirements that the Spark project must fulfill. Functional requirements explain what the system should do, while non-functional requirements describe how the system should perform, its usability, and reliability.

Functional Requirements:

1. The system must accept Spark source code as input.
2. The system must perform lexical analysis to break code into tokens.
3. The system must perform syntax analysis (parsing) to check grammar rules.
4. The system must perform semantic analysis to detect errors.
5. The system must generate intermediate code (Three Address Code).
6. The system must generate target code from the intermediate code.
7. The system must display error messages clearly.
8. The system must support basic Spark language features such as variables, arithmetic operations, comments, and print statements.

Non-Functional Requirements:

1. The system should be easy to use for beginners.
2. The system should be modular and maintainable.
3. The system should run efficiently using Node.js.
4. The system should be scalable to add new features in the future.

2.1.2 Entity Relationship Diagram

Spark is a programming language and not a database. No ERD provided in here.

2.2 Use of Modern Tools

This section explains the tools and technologies used to develop the Spark compiler. Modern tools improve development speed, reduce errors, and make the system easier to maintain. They also help in testing and future extensions.

Tools and Technologies:

1. **Node.js:** It is main platform for implementing the compiler. It enables us to run JavaScript code outside website and creates server. It supports file handling, string processing, and modular coding. It also makes testing and running compiler easy.
2. **Visual Studio Code (VS Code):** VS Code is a code editor with debugging and syntax highlighting. It supports Node.js extension to make development faster.
3. **Git & GitHub:** This is for the version control for tracking project changes. It enables collaboration and easier maintenance.
4. **Diagram Tools (draw.io):** It is used to create flowcharts and diagram for the system architecture.
5. **Testing tools:** Manual testing of Spark program. It helps to verify correctness of error messages, parsing and code output.

Chapter 3

Implementation and Results

This chapter details the step-by-step implementation process of the project. It presents the development phases, key features, and the results obtained from testing and evaluation.

3.1 Implementation

The Spark project is a web-based compiler where users can write, upload, and run code with the ".spark" extension. The project is built using JavaScript, Node.js, Express.js, and EJS for the front-end templates and JavaScript for the backend. The main focus of the implementation was to support variable and constant declarations, boolean values, conditional statements, loops, arithmetic expressions, and error handling and post increment.

The implementation is divided into several phases, which work together to convert user code into an executable form. The overall process is:

Code → Tokenization → Parsing (AST) → Semantic Analysis → Execution

3.1.1 Tokenization

The first step is the Lexer. This component scans the raw string of code and breaks it down into small, meaningful units called Tokens. The Lexer uses a cursor-based approach. It maintains a cursor (the current position in the text) and a line counter. As it loops through the source code, it evaluates characters one by one or in small groups (lookaheads) to decide which category a piece of text belongs to.

When the Lexer encounters a letter, it collects all following letters and digits to form a word. It then checks if this word exists in the reservedKeyWords list (like input, if, or while). If it's not a keyword or a boolean (true/false), it is labeled as an IDENTIFIER, which represents a variable name.

If a character is a digit, the Lexer begins building a NUMBER token. It is designed to handle floating-point numbers by tracking the dotCount. If more than one decimal point is found (e.g., 10.5.2), it provides a "number format" error. Additionally, it prevents illegal variable naming by checking if a word starts with a digit (e.g., 1variable), or any special character rather than underscore.

Strings are written between double quotes ("). The lexer can understand special characters. If it sees a backslash (\), it knows the next character is special—like \n means a new line and \t means a tab—instead of just normal text. It stores the string token as STRING.

The lexer looks for operators in a certain order to avoid mistakes. First, it checks for two-character operators (like ==, !=, or +=). If it finds one, it moves forward by two characters. If not, it looks for single-character operators (like + or =) and symbols (like { or ;).

The lexer recognizes single-line comments (//) and multi-line comments (/* */) and skips them. When it finds a '/', it checks for the next character. If it is another '/', it considers it as a single-line comment and ignores everything from the first '/' until the end of the line. If the next character is *, it recognizes it as a multi-line comment and continues until it finds */. If */ is not found, the lexer considers it an error and stores a error

message in the error variable.

If the lexer finds a character that it doesn't recognize, or a string or comment that starts but never ends (unterminated), or invalid variable name, it creates a clear error message with the exact line number. This helps the user quickly find and fix problems in Spark code.

Using a centralized addToken function makes sure every token has three important pieces of information type, value, and line number for error msg. This structure is important for the next phase (the Parser) which uses these tokens to build the program's logical structure.

3.1.2 Parser (AST)

In parser phase it takes the list of tokens from the Lexer and turns them into a tree-like structure called the Abstract Syntax Tree (AST). This tree shows how different parts of the code are connected. For example, which value belongs to which variable or which code block belongs to a loop.

The parser uses Recursive Descent Parsing. It works like a ladder or waterfall. It starts with the most complex part (like a full statement). It breaks it statements into smaller parts (expressions, then terms, then basic values). Two main helper functions are used: peek() and eat(). Peek function looks at the next token without moving, and eat function consumes the token and moves the cursor forward.

To ensure math is evaluated correctly for example, 5 + 2 * 10 equals 25 and not 70 the parser uses an expression ladder. At the primary level, it handles the most basic units: numbers, variable names, booleans, and parentheses. When it sees an opening bracket, it recursively evaluates everything inside first, giving it top priority. Next, using parseMultiplicative() it groups multiplication, division, and modulo operations (*, /, %) before handling addition and subtraction (+, -). Finally, it evaluates logical and comparison operations like ==, !=, &&, and ||. This layered approach ensures that arithmetic is always computed before comparisons or logical checks.

Handling complex structures like if-else statements and while loops require the parser to manage blocks (code inside { }). For conditional statements, when the parser sees an if keyword, it creates an IfStatement node containing a test (the condition) and a consequent (the block to run). Any following elif or else is linked to the alternate property of the previous node, which form a chain of conditions. For loops, a WhileStatement node stores the loop condition and a body array containing all statements to repeat. Break and continue are handled as simple control flow nodes. It records their type and line number so the Evaluator knows when to exit or continue a loop.

The parser knows the difference between creating a variable and changing it. When it sees input or const, it expects a variable name and optionally a value. For const, a value must be given right away, or it gives an error. If a line starts with a variable name followed by = or +=, the parser considers it as an assignment. It also understands shorthand updates like a++ or b-- as UnaryExpression, so these changes to existing variables work correctly.

The parser is designed to be helpful. If it expects a semicolon (;) or a closing bracket (}). If it doesn't find it, it doesn't crash. Instead, it uses the expect() function to record a clear error message with the line number and what is missing, then continues parsing the rest of the code.

3.1.3 Semantic Analysis

The Analyzer works as the logic checker of the project. While the Parser checks if the code follows the correct grammar, the Analyzer checks if the code actually makes sense. It does this by going through the AST and making sure variables are used correctly, data types match, and control flow is valid.

One of the most important parts of the analyzer is handling scopes. The implementation uses a stack of "scope maps". It manages where variables can be used. There are global and local scopes. When the analyzer enters a block, such as an if statement or a while loop, it creates a new scope and pushes it onto the stack. Any variables declared inside that block stay in that local scope. When a variable is used, the analyzer searches from the top of the stack (the most recent scope) down to the global scope. This lets variables with the same name exist in different places and also stops a program from using a local variable outside its block. The analyzer also checks for errors, such as redeclaring a variable in the same scope, and it ensures that const variables are given a value when they are declared.

Spark is designed to work with different data types like numbers, booleans, and strings. The analyzer uses a function called inferType to guess the result type of an expression. For math operations, it makes sure operators like * or / are only used with numbers. If someone tries to multiply a string by a number, the analyzer shows an error. For if and while statements, the analyzer checks that the condition gives a boolean result. It also makes sure comparison operators (like == or <=) compare values of the same type. The analyzer also protects constants. If a program tries to change a const variable, it stops the action and shows a "Cannot reassign constant" error.

A special feature of the Spark analyzer is the use of loopDepth variable to track whether the code is inside a loop. When entering a while loop, loopDepth increases. Again when leaving the while loop, it decreases. This helps the analyzer check that break and continue are only used inside loops. If they appear outside, it reports an error. The analyzer also handles string interpolation in print statements for example, "Value: ${a}". It uses patterns to find variable names inside ${} and checks if those variables are already declared before allowing them to be printed.

To help developers understand how their code is being processed, the analyzer creates table snapshots. Every time a variable is declared or changed, a snapshot of the current state of all scopes is saved. This allows the Spark web interface to show a visual view of the symbol table, which is very helpful for learning and debugging.

3.1.4 Evaluator (Execution)

The Evaluator is the final and most important part of the Spark project. While earlier stages focus on understanding and checking the code, the Evaluator actually runs it. It takes the verified AST and goes through each part to produce the final output. It also manages memory and program flow in real time.

To store variables during execution, the Evaluator uses a scope stack (an array of objects). When a variable is created using input or const, it is added to the current scope along with extra information like if it is constant. When a variable is used, the Evaluator looks for it using a function called getVariable(). This function searches from the most recent scope down to the global scope. For assignments, it handles both simple (=) and compound ones (like += or *=) by getting the current value, performing the operation, and updating the variable in the correct scope. It also does a final safety check. If a const variable is changed during execution, it stops and shows a "Runtime error."

For expression evaluation the main logic of the evaluator is in function called evalExpr(). Since Spark is a web-based language and input can come in different formats, the Evaluator makes sure data types are handled correctly. The parseLiteral() function converts tokens into real JavaScript types, so "10" becomes a Number and "true" becomes a Boolean. The evalBinary() function performs math operations like +, -, *, /, and %. It also checks for errors such as division by zero. The Evaluator also handles logical operations (&&, ||) and comparisons (like == or <=). It uses short-circuit logic, which means if the first part of an || is true, it does not check the second part. As a result, it will save time of compilation.

The Evaluator controls how the program moves between different blocks of code, such as loops and conditionals. For if-else, the executeIf() function checks the condition; if it is true, it runs the main block,

otherwise it checks elif or runs the else block. Each block runs in a new local scope. So, variables inside the if block do not affect the outside code. For while loops, the evaluator uses a loop that keeps running and checks the condition in every iteration. When the condition becomes false, the loop stops. To handle break and continue, the evaluator uses special flags. When a break statement is executed, a breakFlag is set to true. It stops the loop immediately. When a continue statement is executed, a continueFlag is set. It skips the rest of the current loop iteration and goes back to check the loop condition again.

The print statement in Spark can do more than just show text. It supports string interpolation. When the evaluator sees a string like "Result: ${a + b}", it uses a pattern to find the expression inside ${}. It then passes that expression to a function called evalInterpolationExpr(), which calculates the value or looks up the variables. The result is inserted back into the string. The final combined text is added to the output that the user sees in the web page.

3.2 Output

The output of the Spark web-based compiler gives real-time feedback to the user. After the code passes through the Lexer, Parser, and Analyzer, the Evaluator runs it and shows the results in a dedicated output area.

When a user runs demo code, the system processes variables and logic step by step. For example, if the user defines input a = 10; and input b = 15.5;, the output area will show the calculated values. If a print statement like print "Values: ${a}, ${b}, ${c}"; is executed, the web interface dynamically updates to show the string with the actual values of the variables after clicking the run button.

Spark provides detailed, multi-stage error reporting. Instead of just saying "Error," the console shows exactly where the problem happened. Such as:

1. Line 1: Invalid identifier '21s' (cannot start with a digit)
2. Line 2: Missing closing '}' after block
3. Cannot reassign constant 'c'
4. Runtime error: Division by zero

In addition to text output, Spark shows a snapshot view of the symbol table at different points in the program. Users can see variable names, their types (Number, Boolean, or String), and current values. This makes the output section a powerful tool for learning and debugging, not just a simple display of text.

3.3 Discussion

The Spark project successfully combines a web-based environment with a functional programming language. By separating the Lexer, Parser, and Evaluator, the system is organized, easy to maintain, and simple to extend.

One of the main challenges was handling operator precedence and nested statements. Using a tree-like structure (AST) allowed Spark to manage complex math expressions and deeply nested if-else blocks without losing track of logic. Features like Boolean support and post-increment/decrement (e.g., a++) make the language simple and easy to understand for users who know JavaScript or C++.

The use of a Recursive Descent Parser ensures that mathematical operations are evaluated correctly by organizing them into levels (Primary → Multiplicative → Additive). Built-in scope management keeps variables inside loops and conditional blocks local. It prevents errors from variable leakage.

Some challenges faced creating string interpolation in print statements and handling break/continue statements inside loops. The Evaluator solves these by using a mini-parser to evaluate expressions inside ${} and flags to manage loop control, allowing the program to run smoothly without crashing.

Also, the file upload system for ".spark" files let users work with full programs, not just small example code. Future improvements could include support for functions and more complex data structures, like arrays, to make the language even more versatile and powerful.

Chapter 4

Engineering Standards and Mapping

This chapter discusses the engineering standards followed throughout the project and maps the project activities to the course and program outcomes. It highlights how the project meets academic and professional requirements.

4.1 Impact on Society, Environment and Sustainability

The Spark project is designed with consideration of its effects on people, society, and the environment. It ensures responsible use of technology and promotes ethical handling of data.

4.1.1 Impact on Life

The Spark compiler helps students, educators, and programmers by providing an interactive online platform to learn and test code. It simplifies understanding of programming concepts, compiler design, and debugging practices. This can improve education, programming efficiency, and problem-solving skills, making coding more accessible to learners worldwide.

4.1.2 Impact on Society & Environment

By being web-based, the project reduces the need for installing heavy software locally, which saves computing resources and reduces e-waste. It supports collaborative learning and knowledge sharing among students and developers. The project contributes positively to society by enhancing programming education and providing an open-access tool for learning compiler design principles.

4.1.3 Ethical Aspects

The project follows ethical principles to ensure user privacy, data security, and intellectual property protection. Sensitive data is handled responsibly using authentication and access control mechanisms, preventing unauthorized access or misuse. Data sources and usage policies are documented clearly, promoting transparency. By following ethical guidelines, the Spark project builds trust, accountability, and responsible use of technology in an educational environment.

4.2 Complex Engineering Problem

The Spark project addresses complex engineering problems by integrating multiple disciplines: formal languages, compiler design, algorithms, data structures, and software engineering practices. It requires careful design decisions, analytical reasoning, and practical implementation.

4.2.1 Mapping of Program Outcome

The project aligns with several Program Outcomes (POs) by connecting theoretical knowledge to practical compiler implementation.

Table 4.1: Justification of Program Outcomes

POs	Justification of Mapping (Project Perspective)
PO 1	Theoretical concepts of formal languages, automata theory, algorithms, and data structures applied to understand the internal working mechanism of a compiler. Applying this knowledge to develop compiler components such as lexical analyzer, parser, and code generator.
PO 2	The project requires to analyze programming language syntax and semantics, identify compiler errors, and apply analytical reasoning to design solutions for parsing, semantic checking, and error recovery problems.
PO 3	Designing and developing the complete compiler architecture by implementing various modules (lexical analysis, syntax analysis, code generation, optimization). The project emphasizes modular design, integration, testing, and performance evaluation.
PO 10	Preparing project reports, documentation, and presentations explaining compiler structure, design decisions, and experimental results. This demonstrate and defend the implementation to peers and faculty through oral and visual presentations.

4.2.2 Complex Problem Solving

Table 4.2: Mapping with complex problem solving.

EP1 Dept of Knowledge	**EP2** Range of Conflicting Requirements	**EP3** Depth of Analysis	**EP4** Familiarity of Issues	**EP5** Extent of Applicable Codes	**EP6** Extent Of Stakeholder Involvement	**EP7** Inter-dependence
✓	✓		✓			

EP1: The compiler design project demands deep understanding of **automata theory, formal language grammar, parsing algorithms, data structures, and machine architecture**. Students must apply mathematical and theoretical computer science concepts to design compiler components like lexical analyzers, parsers, and code generators.

EP2: The project involves balancing **efficiency, accuracy, optimization, and code generation speed**. For example, selecting appropriate parsing techniques (LL vs. LR) that trade between computational efficiency and grammar complexity.

EP4: Unfamiliar challenges arise, such as **handling ambiguous grammars, semantic errors, or integrating compiler tools (Lex, Yacc, Flex, Bison)**. Documentation and research resources beyond classroom instruction being consulted, fostering independent learning and adaptability.

4.2.3 Complex Engineering Activities

The Spark project involves multiple engineering activities requiring planning, collaboration, and technical decision-making.

Table 4.2: Mapping with complex engineering activities.

EA1	EA2	EA3	EA4	EA5
✓	✓			

EA1: Appropriate programming languages (JavaScript, Node.js), libraries, and IDEs are selected to implement compiler modules such as lexical analysis, parsing, semantic checking, and code generation. Suitable data structures and algorithms are also chosen to ensure efficient performance and maintainable design.

EA2: Team collaboration is applied by dividing responsibilities for different compiler modules. Work is integrated, debugging is performed collectively, and documentation and presentations are prepared, reflecting real-world software engineering teamwork and coordination.

Chapter 5

Conclusion

This chapter provides a summary of the Spark Compiler project, highlights its limitations, and discusses possible directions for future work. It reflects on the achievements of the project and areas where it can be enhanced.

5.1 Summary

The Spark project is a web-based compiler that allows users to write, upload, and run Spark programs online. The system follows a structured workflow where code is first read, then tokenized, parsed into an abstract syntax tree (AST), analyzed semantically for errors, and finally executed to produce results. It supports variable and constant declarations, arithmetic and boolean expressions, operator precedence, conditional statements (if, elif, else), loops (while and nested loops), and statements such as break, continue, and post-increment/decrement operations.

The compiler is capable of detecting and reporting errors at each phase, including lexical, syntax, semantic, and runtime errors. This helps users understand the cause of errors and correct their code efficiently. The web interface, built using JavaScript, Node.js, Express.js, and EJS. It provides a platform to write and execute code instantly, making it highly interactive and user-friendly.

Overall, the Spark project demonstrates a practical understanding of compiler design, software engineering principles, and web-based system development. It combines theoretical knowledge of programming languages, parsing techniques, and execution models, which provides an educational and experimental environment. The project provides an accessible platform for users to learn programming, test ideas, and explore compiler behavior in real time.

5.2 Limitation

Despite its functionality, the Spark compiler has several limitations. Currently, it supports only basic language features, which means advanced constructs like functions, arrays, objects, or user-defined data types are not implemented. Error handling is included at each phase. But error messages could be more descriptive, especially for complex or nested expressions. It would help in debugging difficult code.

Performance is optimized for small to medium programs. But very large programs may experience slower execution because the system processes all phases sequentially. Additionally, the web interface is functional but it lacks advanced features such as syntax highlighting, code auto-completion, debugging tools, or customizable themes. These limitations highlight areas where the compiler can be extended and can be improved to offer better performance, usability, and user experience.

5.3 Future Work

The Spark project has significant potential for future improvement and expansion. The platform can be extended to support more advanced language features such as functions, arrays, objects, and other complex data structures. Adding advanced debugging tools such as syntax highlighting, error tracing, code suggestions, and step-by-step execution will improve the learning and coding experience.

Performance can be enhanced by optimizing memory usage and execution speed. It will allow the system to handle larger programs efficiently. The web interface can be improved with better file management, customizable themes, and user-friendly design, which will make it more user-friendly to users.

Future enhancements could also include integration with educational platforms such as tutorials, exercises, and collaborative coding tools to make the compiler more interactive and supportive for learning. Additionally, multi-language support or a hybrid coding environment can be added. This will allow the system to be used by more people and support more programming languages.

With these improvements, the Spark compiler can become a robust, interactive, and educational tool, combining practical learning with real-time code execution. It has the potential to support programming education, research, and experimentation while providing a scalable and sustainable platform for the future.

References